

Pitch: Aufgabenblatt 2

Probabilistisches Modell für finale Position

Das Aufgabenblatt verlangte einen Sim2Real Vergleich und ein probabilistisches Endpunkt-Modell.

Das Hauptproblem war nicht das Schreiben eines Fortbewegungs-Scripts.

Es war die genaue Erfassung der Position, weil jede spätere Entscheidung darauf basierte.

Dazu habe ich mir folgende Probleme angeschaut.

Ablaufplan

- Problem 1: Positionsbestimmung
- Problem 2: Konsistenz in der Messung
- Problem 3: Simulation vs reale Bewegung
- Problem 4: Probabilistisches Modell

Positionsbestimmung

Problem

Die Aufgabe verlangt eine finale Position, aber der TurtleBot übergibt keine saubere absolute Endposition.

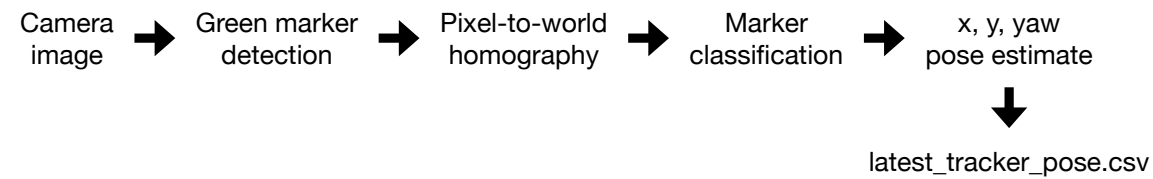
Entscheidung

Odometrie ist relativ und hat potentiellen Drift, deswegen habe ich eine externe Kamera als Tracker genutzt.

Konsequenz

Dies ermöglichte einen Vergleich von Simulationseckpunkten und realen Eckpunkten.

Pipeline



Stephanus Park, 12.05.26, MII-AMR-M

3

Zunächst musste folgende Frage beantwortet werden:
Wie kann ich wissen, wo der TurtleBot am Ende wirklich ist?

Odometrie war nützlich, aber nicht genug für eine absolute Positionsbestimmung.

Ich entschied mich für eine Overhead-Kamera als Tracker und drei grüne Markierungen auf dem Roboter.

Der Tracker konvertiert anhand einer Homographie-Matrix die Pixel in reale Meter.

Der Positionsschätzer klassifiziert die Markierungen, berechnet x/y/yaw und speichert sie in einer CSV.

Das garantierte mir ein übergreifendes Koordinatensystem für die finalen Endpunkte.

Konsistenz in der Messung

Problem

Kleine manuelle Abweichungen bei der Startposition können als Modellierungsfehler interpretiert werden.

Toleranzgrenzen

Positionstoleranz:	0.04 m
Yaw-Toleranz:	4 Grad
Stabile Zeit:	1 Sekunde
Erfasste Markierungen:	3

Entscheidung

Einführung einer Toleranzgrenze bezüglich Position und Winkel beim Start.

Konsequenz

Probabilistisches Modell vermeidet Vermischung von realen Bewegungsabweichungen mit inkonsistenter Startpositionierung.

Stephanus Park, 12.05.26, MII-AMR-M

4

Das nächste Problem war Reproduzierbarkeit.

Wenn die Startposition sich verändert, dann beeinflusst das die Statistik bezüglich der Endpunkte.

Ich fügte eine Toleranzgrenze hinzu, die den Start von realen Läufen kontrollierte.

Es mussten drei Markierungen gefunden werden, unter 4 cm Positionsabweichung sein und weniger als 4 Grad Winkelunterschied zur festgelegten Startposition vorliegen.

Die Datenerhebung war somit langsamer, aber dafür gründlicher.

Reale und simulierte Bewegung

Problem

Derselbe Bewegungsbefehl erzeugt in Simulation und Realität unterschiedliche Endpositionen.

Entscheidung

Zerlegung in wiederholbare Bewegungsprimitiven.

Konsequenz

Aus den wiederholten realen Primitiven konnten Mittelwert, Streuung und Drift für das spätere probabilistische Modell bestimmt werden

Bewegungsprimitive

<i>F30</i>	30 cm forward	<i>CW45</i>	45 deg clockwise rotation
<i>F50</i>	50 cm forward	<i>CCW45</i>	45 deg counterclockwise rotation
<i>CW90</i>	90 deg clockwise rotation	<i>CCW180</i>	180 deg counterclockwise rotation
<i>CCW90</i>	90 deg counterclockwise rotation		

Stephanus Park, 12.05.26, MII-AMR-M

5

Derselbe Befehl wurde in der Simulation und auf dem realen Roboter unterschiedlich ausgeführt.

In der Simulation kann das Script stoppen, sobald die Odometrie die Zieldistanz erreicht.

Dahingegen müssen auf dem Roboter die Experimente über die Zeit oder Bewegungsprimitive gesteuert werden.

Ich entschied mich dazu, einzelne Bewegungsprimitive zu nutzen, da die finale End-Route aus simplen wiederkehrenden Teilbewegungen bestand: Lineare Vorwärtsbewegung und Rotation.

Diese Teilbewegungen konnte ich einzeln mehrfach ausführen, messen und daraus Mittelwert und Streuung bestimmen.

Dadurch basiert mein späteres Modell nicht auf idealer Kinematik, sondern auf dem tatsächlich gemessenen Verhalten des Turtlebots.

Probabilistisches Modell

Problem

Keine Implementierung von SLAM oder vollumfängliche Trajektionsunsicherheit.

Entscheidung

Nutzung eines simplen empirischen Gaußschen Modells.

Konsequenz

Grobe Vorhersage der Endposition, dafür aber einfacher zu erklären und basiert direkt auf erhobenen Daten.

For each primitive a :

$$\Delta p_{\text{local}} \sim N(\mu_a, \Sigma_a)$$

$$\Delta yaw \sim N(\mu_{yaw,a}, \sigma_{yaw,a})$$

$$\Delta p_{\text{world}} = R(yaw) \cdot \Delta p_{\text{local}}$$

$$p \leftarrow p + \Delta p_{\text{world}}$$

$$yaw \leftarrow \text{normalize}(yaw + \Delta yaw)$$

**Pseudocode, wie aus
Primitiven eine Endpose
berechnet wird**

Stephanus Park, 12.05.26, MII-AMR-M

6

Ich habe mich für ein simples empirisches Modell entschieden,
weil die Aufgabe eine grobe Vorhersage der Endposition verlangt hat und kein vollständiges SLAM- oder Trajektorienmodell.

Die Idee ist: Ich lerne für jedes Bewegungsprimitive aus den realen Messdaten eine lokale Verteilung.

Für die Translation sample ich eine lokale Verschiebung Δp_{local} aus einer zweidimensionalen Gaußverteilung.
Für die Rotation sample ich separat eine Änderung Δyaw aus der gemessenen Yaw-Verteilung.

Wichtig ist der nächste Schritt:

Diese lokale Verschiebung gilt im Roboter-Koordinatensystem.

Damit sie zur globalen Position passt, rotiere ich sie mit der aktuellen Orientierung des Roboters:

$$\Delta p_{\text{world}} = R(yaw) \cdot \Delta p_{\text{local}}.$$

Danach aktualisiere ich die Pose:

Die Position wird um Δp_{world} verschoben,
und yaw wird um Δyaw verändert.

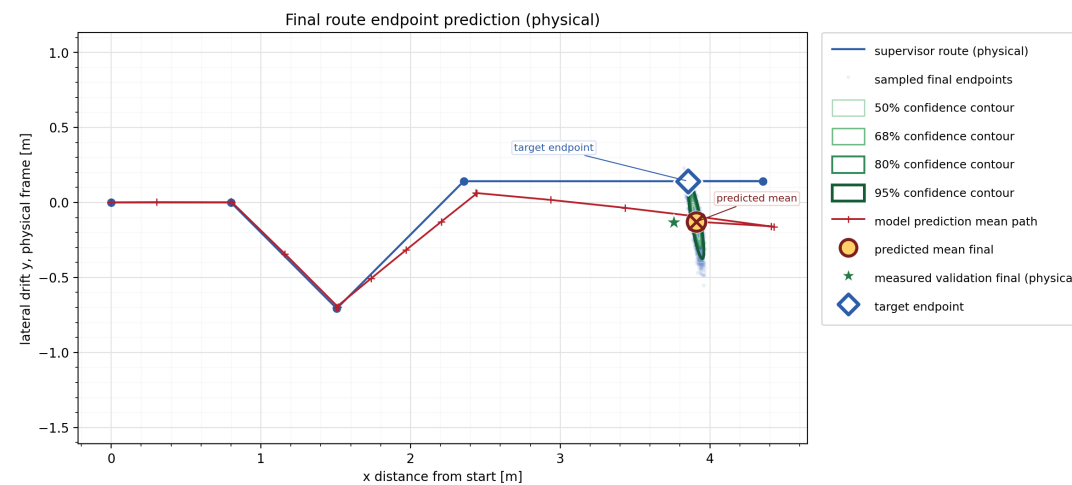
Das Modell bleibt bewusst einfach:

Ich lerne keine vollständige gemeinsame Verteilung über x, y und yaw,

sondern Translation und Yaw getrennt.

Aber es reicht, um aus gemessenen Bewegungsprimitiven viele mögliche Endpositionen der Route zu simulieren.

Evaluation des Modells



Stephanus Park, 12.05.26, MII-AMR-M

7

Die Abweichung von ungefähr 15 cm ist für mich kein reiner Messfehler, sondern zeigt die Grenze meines Modellierungsansatzes.

Ich habe die Endposition aus einzeln gelernten Bewegungsprimitiven vorhergesagt.

Diese Primitiven waren in den Einzelversuchen relativ stabil, aber bei der langen End-Route werden viele kleine Fehler hintereinander zusammengesetzt.

Dadurch können sich systematische Abweichungen aufsummieren.

Das Modell war deshalb zu optimistisch und hat die Unsicherheit der gesamten Route unterschätzt.

Die 95%-Ellipse basiert auf den Varianzen der gelernten Primitiven.

Diese Daten stammen aus kontrollierten Einzelbewegungen.

In der langen Route treten zusätzliche Effekte auf:

- Kopplung zwischen Rotationsfehlern und späteren Vorwärtsbewegungen,
- leichte Start-/Stop-Unterschiede,
- Bodenkontakt,
- Schlupf und Odometrie-basierte Ausführung.

Diese Effekte wurden im einfachen Modell nicht vollständig abgebildet.

-- --

Ich habe Monte-Carlo-Sampling verwendet, weil sich die Unsicherheiten der einzelnen Bewegungsprimitive über die gesamte Route fortpflanzen.

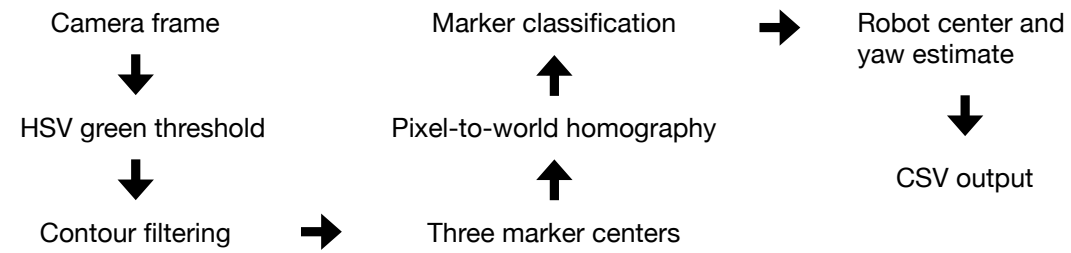
Besonders Rotationsfehler wirken sich auf alle folgenden Vorwärtsbewegungen aus.

Dadurch addieren sich die Fehler nicht einfach linear.

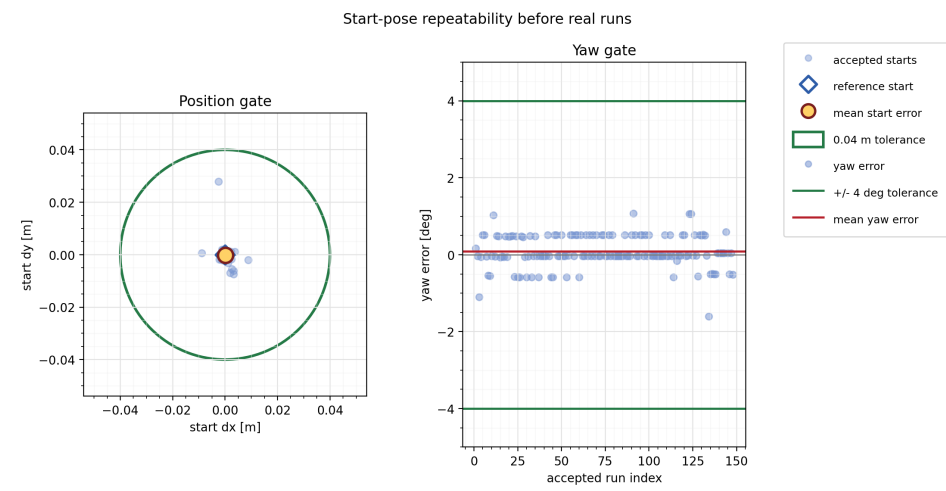
Mit Monte Carlo konnte ich viele mögliche Ausführungen der Route simulieren und daraus die erwartete Endposition sowie die Unsicherheitsregion bestimmen.

Vielen Dank!

Appendix: Camera pipeline

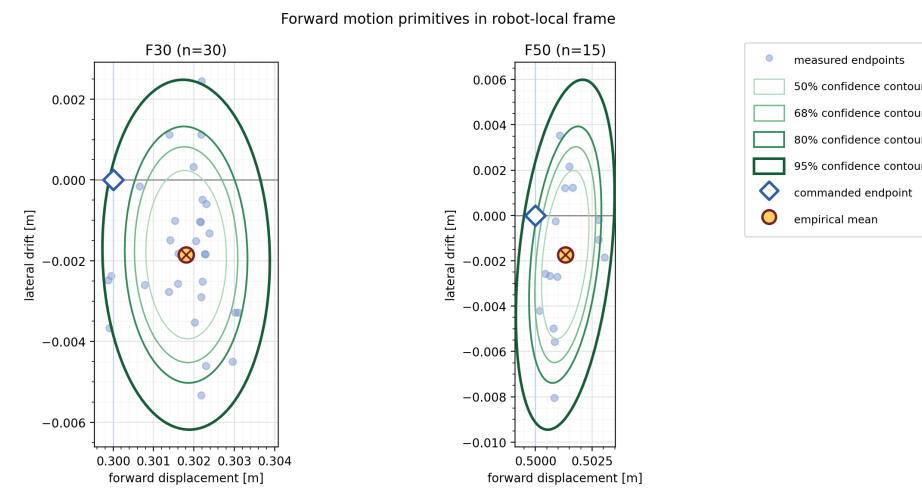


Appendix: Toleranzgrenzen



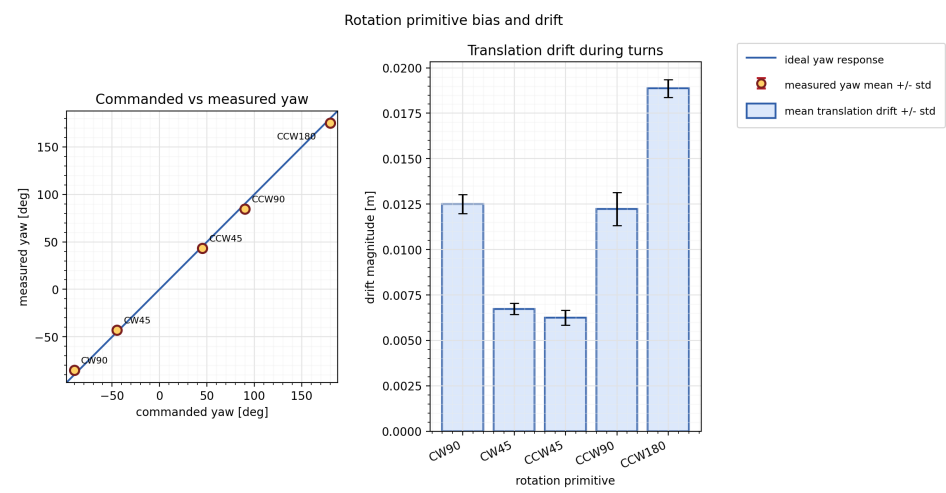
Stephanus Park, 12.05.26, MII-AMR-M

Appendix: Bewegungsprimitive



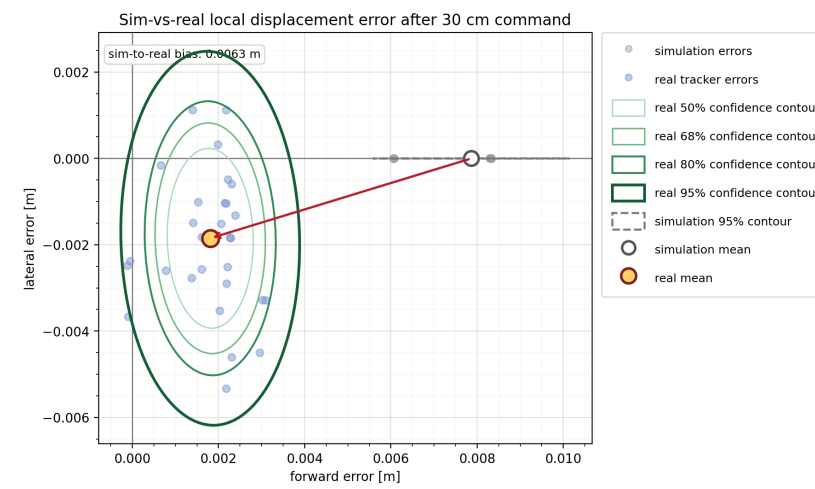
Stephanus Park, 12.05.26, MII-AMR-M

Appendix: Rotations-Bias



Stephanus Park, 12.05.26, MII-AMR-M

Appendix: Sim2Real Vergleich



Stephanus Park, 12.05.26, MII-AMR-M